

MODULE 03 · DOMAIN 1 - DATA INGESTION AND TRANSFORMATION

Streaming ingestion

Kinesis Data Streams, Amazon Data Firehose, Amazon MSK, and Managed Service for Apache Flink.

DOMAIN 1 of 4 EXAM WEIGHT 34% MODULE 3 of 27

Streaming is not about how fast data **arrives**. It is about how fast someone needs an **answer**.

The anatomy of a stream

01

Producer

Writes records.
An app, an
agent, a service.

02

Stream

An ordered,
durable log held
for a retention
window.

03

Consumer

Reads at its own
pace, tracking its
own position.

04

Processing

Optional. Filter,
aggregate, join,
enrich.

05

Destination

A lake, a
warehouse, a
search index, an
alert.

ON THE EXAM

The stream stores data. That is what separates it from a message queue.

Stream or queue?

AMAZON SQS

A work queue

- A message is consumed then deleted
- One consumer per message, by design
- No replay after processing
- Perfect for decoupling work items

KINESIS DATA STREAMS

A durable log

- Records persist for the retention window
- Many consumers read independently
- Replay by rewinding position
- Ordering guaranteed within a shard

THE TIEBREAKER

Multiple independent consumers, replay, or strict ordering means a stream. Otherwise a queue is simpler and cheaper.

The four streaming services

SERVICE	IN ONE SENTENCE
Kinesis Data Streams	A managed, shard-based durable log you read from yourself
Amazon Data Firehose	Managed delivery to a destination. No consumers, no code
Amazon MSK	Managed Apache Kafka, for Kafka APIs and Kafka ecosystems
Managed Service for Apache Flink	Stateful stream processing. Windows, joins, aggregations

ON THE EXAM

Firehose delivers. Kinesis and MSK transport. Flink computes. Most questions are asking which of those three verbs you need.

A stream is a set of **shards**. Every limit that matters is a per-shard limit.

Per-shard limits

DIRECTION	LIMIT
Write	1 MB per second, or 1,000 records per second
Read, standard	2 MB per second, shared across all consumers
Read, transactions	5 GetRecords calls per second per shard
Read, enhanced fan-out	2 MB per second per consumer, not shared
GetRecords response	Up to 10 MB, or 10,000 records, per call

ON THE EXAM

Whichever limit binds first is the one that throttles you. Small frequent records hit 1,000 per second long before 1 MB.

Provisioned or on-demand?

PROVISIONED

You choose the shard count

- Pay per shard-hour, used or not
- Cheapest at steady, known throughput
- You resize with UpdateShardCount
- Throttles the moment you exceed it

ON-DEMAND

Kinesis chooses

- Pay per GB written and read
- Best for unpredictable or spiky load
- Scales automatically as traffic grows
- Costs more at high steady throughput

THE TIEBREAKER

Unknown, spiky or new workloads start on-demand. Steady, measured, high-volume workloads are cheaper provisioned.

On-demand, by the numbers

- ◆ **New streams start** at 4 MB per second write, 8 MB per second read.
- ◆ **Scales to 10 GB/s write** and 20 GB/s read in N. Virginia, Oregon and Ireland.
- ◆ **200 MB/s write elsewhere**, unless you raise it through Support.
- ◆ **Provisioned shard quota** is now 20,000 per account in those same Regions.
- ◆ **Scaling is not instant**. A sudden tenfold spike can still throttle briefly.

ON THE EXAM

These limits rose sharply in late 2024 and 2025. Older study material quotes 200 MB/s as the ceiling everywhere.

Partition keys, and the hot shard problem

- ◆ The **partition key** decides which shard a record lands on, by hash.
- ◆ **Same key, same shard**, which is what preserves ordering per key.
- ◆ A **key that dominates** sends most traffic to one shard. That shard throttles.
- ◆ **Adding shards does not help**. The hot key still hashes to one of them.
- ◆ **Fix the key**. Higher cardinality, or add a random suffix to spread it.

ON THE EXAM

Throttling on some shards while others sit idle is the signature of a hot partition key.

Resharding and retention

- ◆ **Split a shard** to add capacity. **Merge two** to remove it.
- ◆ **UpdateShardCount** does the arithmetic for you in provisioned mode.
- ◆ **Retention defaults to 24 hours**, extendable to 7 days.
- ◆ **Long-term retention** reaches 365 days, at additional cost.
- ◆ Retention is what makes replay possible. No retention, no replay.

ON THE EXAM

A question about reprocessing the last three days is asking whether retention was extended beyond the 24 hour default.

Standard consumers or enhanced fan-out?

STANDARD

Shared 2 MB per second

- All consumers share one shard's read capacity
- Polling model, roughly 200 ms latency
- Free. No per-consumer charge
- Fine for one or two consumers

ENHANCED FAN-OUT

2 MB per second each

- Every consumer gets its own dedicated pipe
- Push model, roughly 70 ms latency
- Charged per consumer-shard hour
- Needed beyond a few consumers

THE TIEBREAKER

Several consumers slowing each other down is the enhanced fan-out signal. So is a hard latency requirement.

Getting data in and out

- ◆ **Kinesis Producer Library.** Batches and aggregates records for throughput.
- ◆ **Kinesis Agent.** A daemon that tails log files and ships them.
- ◆ **PutRecord and PutRecords.** The plain SDK path, one or many at a time.
- ◆ **Kinesis Client Library.** Handles checkpointing and shard rebalancing for consumers.
- ◆ **KCL 1.x and KPL 0.x** reached end of support on 30 January 2026.

ON THE EXAM

The KCL exists so you do not hand-write checkpointing. Rewriting it yourself is rarely the exam answer.

Tuning Lambda as a stream consumer

- ◆ **Batch size.** How many records per invocation. Bigger is cheaper, and slower to react.
- ◆ **Batch window.** Wait up to N seconds to fill a batch before invoking.
- ◆ **Parallelisation factor.** Up to 10 concurrent invocations per shard.
- ◆ **One poison record** blocks its shard until it expires. The batch retries forever.
- ◆ **Bisect batch on error** plus an on-failure destination isolates the bad record.

ON THE EXAM

A stalled consumer and a climbing iterator age, with no obvious load problem, is usually one failing record.

A ride-hailing company streams vehicle location updates into Amazon Kinesis Data Streams in provisioned mode with twelve shards. Records use the city name as the partition key, and the company operates in eleven cities.

The operations team reports intermittent write throttling. CloudWatch shows two shards consistently at their write limit while the other ten average under fifteen percent utilisation. Total ingest across the stream is well under twelve megabytes per second.

What should the data engineer do?

- ☐ A Increase the shard count from twelve to twenty four
- ☐ B Switch the stream to on-demand capacity mode
- ☐ C Change the partition key to include the vehicle identifier as well as the city
- ☐ D Enable enhanced fan-out for the consumer applications

◆ PAUSE AND DECIDE

What should the data engineer do?

- A

 Double the shard count
The two busiest cities still hash to one shard each. You now pay for twenty four shards and throttle identically.
- B

 Switch to on-demand
On-demand splits hot shards over time, so it partly helps. It also costs more and treats a key design fault as a capacity fault.
- C

 Add the vehicle ID to the partition key
Turns eleven keys into thousands, spreading evenly across shards. Addresses the actual cause.
- D

 Enable enhanced fan-out
Enhanced fan-out is a read-side feature. The problem stated is write throttling.

THE SIGNAL

Two shards saturated while ten idle, at low total throughput, is a partition key problem every time.

Firehose has **no consumers**. You choose a destination and it handles everything else.

What Firehose does on the way

- ◆ **Buffers** by size and time, then delivers the batch.
- ◆ **Transforms** with a Lambda function, if you attach one.
- ◆ **Converts format**, JSON to Apache Parquet or ORC, without code.
- ◆ **Compresses**. GZIP, ZIP or Snappy. Only GZIP if loading into Redshift.
- ◆ **Partitions dynamically**, routing records to S3 prefixes by field value.

ON THE EXAM

Format conversion and dynamic partitioning mean Firehose can build a well-organised lake with no ETL job at all.

Where Firehose can deliver

DESTINATION	NOTES
Amazon S3	The common case. Format conversion and dynamic partitioning apply
Apache Iceberg tables and S3 Tables	Supports insert, update and delete operations
Amazon Redshift	Goes to S3 first, then Firehose issues a COPY command
Amazon OpenSearch Service	Including OpenSearch Serverless
Snowflake	Exactly-once delivery. One stream per table
Splunk and HTTP endpoints	Datadog, New Relic, Dynatrace, MongoDB and others

ON THE EXAM

The Redshift path is not direct. Knowing it lands in S3 first explains several exam behaviours.

Buffering, and the latency it buys

- ◆ **Buffer size** 1 to 128 MB for Amazon S3. **Buffer interval** 0 to 900 seconds.
- ◆ **Whichever fills first** triggers delivery.
- ◆ **Zero buffering** delivers in roughly five seconds. Available since December 2023.
- ◆ **Not available with dynamic partitioning**, which buffers per partition.
- ◆ **Smaller buffers mean more, smaller files.** The small file problem, created on purpose.

ON THE EXAM

Firehose is described everywhere as near real time. With zero buffering it is closer to real time than most material admits.

Kinesis Data Streams or Firehose?

DATA STREAMS

You need control

- Custom consumer logic
- Multiple independent consumers
- Replay from a retention window
- Sub-second processing latency

DATA FIREHOSE

You need delivery

- Land it somewhere, managed end to end
- No code, no shards, no consumers
- Format conversion and partitioning built in
- Seconds of latency is acceptable

THE TIEBREAKER

They are often used together. Data Streams as the source, Firehose as one of its consumers.

When Firehose delivery fails

- ◆ **Firehose retries** for up to 24 hours, then discards the data.
- ◆ **Configure S3 backup** so failed or skipped records are written somewhere.
- ◆ **Lambda transformation failures** can be sent to a separate error prefix.
- ◆ **Delivery is at-least-once**, so duplicates are possible on retry.
- ◆ **Except** to Amazon S3, Apache Iceberg tables and Snowflake.

ON THE EXAM

Twenty four hours of retry is generous and finite. A destination down for a weekend loses data without S3 backup.

Streaming straight into Redshift

- ◆ **Redshift streaming ingestion** reads Kinesis Data Streams or Amazon MSK directly.
- ◆ **No S3 staging.** Data lands in a materialized view, refreshed from the stream.
- ◆ Latency in seconds, rather than the minutes a Firehose and COPY path takes.
- ◆ Contrast with Firehose to Redshift, which always writes to S3 first.
- ◆ The materialized view is where you apply SQL transformations.

ON THE EXAM

Two paths from a stream into Redshift. Firehose stages through S3; streaming ingestion does not.

A media company streams clickstream events into Amazon S3 for analysis with Amazon Athena. Events arrive as raw JSON, one object every thirty seconds, into a single prefix with no partitioning. The pipeline is a Kinesis Data Streams stream read by an AWS Lambda function that writes objects to S3.

Analysts report that Athena queries filtering on a single day scan the entire dataset and are becoming expensive. The team also maintains the Lambda function and wants less to operate.

Which change addresses both the query cost and the operational burden?

- | | |
|--|--|
| <p>A Add a scheduled AWS Glue job to convert the JSON to Parquet and rewrite it partitioned by date</p> | <p>B Replace the Lambda consumer with an Amazon Data Firehose stream using format conversion and dynamic partitioning</p> |
| <p>C Increase the Lambda function's memory so it can write larger, better-organised objects</p> | <p>D Move the data into Amazon Redshift and have analysts query it there instead</p> |

◆ PAUSE AND DECIDE

Which change addresses both the query cost and the operational burden?

A Add a scheduled Glue job

Fixes query cost properly, and adds a second component to operate. They asked for less to maintain, not more.

B Firehose with conversion and dynamic partitioning

Converts to Parquet and partitions on arrival, and removes the Lambda function entirely. Both requirements, one change.

C More Lambda memory

Memory affects speed and cost of the function. It does nothing about format, partitioning or maintenance.

D Move to Redshift

Solves the query cost by replacing the whole analytics approach, and adds a warehouse to run.

THE SIGNAL

Option A is the right fix from module two, offered where a managed feature does the same job with nothing to operate.

MSK is **Apache Kafka**. The exam question is almost never Kafka itself. It is which MSK.

Kafka vocabulary, translated

- ◆ **Topic.** A named stream. Roughly a Kinesis stream.
- ◆ **Partition.** The unit of parallelism and ordering. Roughly a shard.
- ◆ **Offset.** A consumer's position in a partition. Roughly a sequence number.
- ◆ **Consumer group.** Consumers sharing the work of one topic.
- ◆ **Broker.** A server holding partitions. Kinesis has no equivalent you manage.

ON THE EXAM

KRaft has replaced ZooKeeper for cluster metadata. Older material still describes ZooKeeper nodes.

The three ways to run MSK

OPTION	CHOOSE IT WHEN
Provisioned, Standard brokers	You need maximum configuration control and custom replication
Provisioned, Express brokers	High throughput. Up to 3x per broker, 20x faster scaling, no storage to size
Serverless	Variable or unknown load. No brokers, no partitions to plan, pay per use

ON THE EXAM

Express is a broker type inside Provisioned, not a fourth cluster type. That naming trips people up.

Kinesis or MSK?

KINESIS DATA STREAMS

AWS-native

- No brokers or partitions to operate
- Deep integration with Lambda, Firehose, Flink
- Simplest answer for a new AWS workload
- Proprietary APIs

AMAZON MSK

Kafka-native

- Existing Kafka applications run unchanged
- The Kafka connector ecosystem
- Migrating an on-premises Kafka estate
- Portability off AWS

THE TIEBREAKER

Existing Kafka code, Kafka APIs, or Kafka tooling means MSK. A greenfield AWS pipeline means Kinesis.

The MSK ecosystem pieces

- ◆ **MSK Connect.** Managed Kafka Connect. Runs source and sink connectors.
- ◆ **MSK Replicator.** Replicates topics between clusters and Regions.
- ◆ **Tiered storage.** Offloads older data to cheaper storage, extending retention.
- ◆ **IAM access control,** plus SASL/SCRAM and mutual TLS as alternatives.
- ◆ **Glue Schema Registry** enforces record schemas across producers and consumers.

ON THE EXAM

MSK Replicator is the answer for cross-Region Kafka disaster recovery, not a self-managed MirrorMaker cluster.

Securing a stream

CONCERN	HOW IT IS HANDLED
Encryption at rest	AWS KMS on Kinesis and MSK. Customer managed keys supported
Encryption in transit	TLS to the service endpoint, and between MSK brokers
Kinesis authorisation	IAM policies on the stream, per action and per consumer
MSK authentication	IAM access control, SASL/SCRAM with Secrets Manager, or mutual TLS
Private networking	Interface VPC endpoints. MSK lives in your VPC by default

ON THE EXAM

MSK runs inside your VPC. Kinesis is a public endpoint unless you add an interface VPC endpoint.

A retailer runs a self-managed Apache Kafka cluster on Amazon EC2 in one Region. Around forty producer and consumer applications use the standard Kafka client libraries, and several use Kafka Connect connectors to move data into Amazon S3.

The platform team spends significant time patching brokers, rebalancing partitions and recovering from broker failures. They want to reduce that work, keep all existing applications running unchanged, and add cross-Region disaster recovery.

What should the company do?

- A Migrate to Amazon Kinesis Data Streams in on-demand mode and rewrite the client applications
- B Migrate to Amazon MSK Provisioned with Express brokers, use MSK Connect, and add MSK Replicator
- C Keep Kafka on EC2 but add an Auto Scaling group and a second cluster with MirrorMaker
- D Migrate to Amazon Data Firehose with Amazon S3 as the destination

◆ PAUSE AND DECIDE

What should the company do?

- A

Kinesis, rewriting the clients

Rewriting forty applications to leave Kafka behind. The question said keep them running unchanged.
- B

MSK Express, MSK Connect, MSK Replicator

Kafka APIs unchanged, brokers and storage managed, connectors managed, cross-Region replication managed. All three requirements.
- C

Auto Scaling plus MirrorMaker

Keeps every operational burden they asked to remove, and adds a second cluster to patch.
- D

Firehose to S3

Firehose is delivery only. Forty applications that publish and subscribe cannot be served by it.

THE SIGNAL

"Keep existing applications running unchanged" is a Kafka API requirement. It eliminates every non-Kafka option immediately.

Transport is not computation. When a question needs **windows, joins or state**, it needs Flink.

What stateful processing means

- ◆ **Aggregation over time.** Counts, sums and averages per window.
- ◆ **Joining streams.** Matching records from two sources on a key.
- ◆ **Pattern detection.** Three failed logins within a minute.
- ◆ **Enrichment.** Adding reference data to each passing record.
- ◆ **Stateless work** like reformatting one record needs no Flink at all.

ON THE EXAM

If each record can be handled entirely on its own, you do not need a stream processor.

Window types

WINDOW	WHAT IT DOES
Tumbling	Fixed, non-overlapping. Every 5 minutes, each record counted once
Sliding	Fixed length, overlapping. Last 5 minutes, evaluated every minute
Session	Grouped by activity, closed after a gap of inactivity

ON THE EXAM

"Every hour, total the last hour" is tumbling. "Continuously, the last hour" is sliding.

Keeping state safe

- ◆ **Checkpoints** are automatic, periodic snapshots for recovery from failure.
- ◆ **Savepoints** are manual snapshots, taken before an upgrade or a change.
- ◆ **Exactly-once** processing semantics are possible because state is checkpointed.
- ◆ **Parallelism** is configurable, and can scale automatically with load.
- ◆ **State grows.** An unbounded key space is a real production failure mode.

ON THE EXAM

Checkpoints recover from crashes. Savepoints let you change the application without losing state.

Two things about the Flink service

- ◆ **Renamed.** Kinesis Data Analytics became Managed Service for Apache Flink in 2023.
- ◆ **Studio notebooks** give interactive Apache Zeppelin against a live stream.
- ◆ **Kinesis Data Analytics for SQL** was a separate, older product.
- ◆ **New applications blocked** from October 2025. Existing ones deleted from 27 January 2026.
- ◆ AWS directs those workloads to Managed Service for Apache Flink.

ON THE EXAM

Availability warning: the SQL variant no longer exists. It still appears in question banks as a correct answer.

Lambda or Flink for stream processing?

AWS LAMBDA

Per-record logic

- Triggered by a batch of stream records
- No state between invocations
- Cheap, simple, nothing to run
- Filtering, reformatting, routing

MANAGED SERVICE FOR APACHE FLINK

Continuous computation

- An always-running application
- Holds state across records and time
- Windows, joins, pattern detection
- More cost, more capability

THE TIEBREAKER

Stateless per-record work is Lambda. Anything needing memory of other records is Flink.

An online payments company streams transaction events into Amazon Kinesis Data Streams. A new requirement is to flag any account that makes more than five transactions in any rolling one minute period, and to raise an alert within two seconds of the fifth transaction.

An existing AWS Lambda function already reads the stream and writes each transaction to Amazon S3. A proposal is to extend that function with the new logic, storing counts in an Amazon DynamoDB table.

What is the most appropriate design?

- ☐ A Extend the existing Lambda function to maintain per-account counts in DynamoDB
- ☐ B Add a Managed Service for Apache Flink application with a sliding window over one minute
- ☐ C Deliver the stream to Amazon S3 with Data Firehose and run an Athena query every minute
- ☐ D Load the transactions into Amazon Redshift and use a materialized view refreshed each minute

◆ PAUSE AND DECIDE

What is the most appropriate design?

- A

Lambda plus DynamoDB counters
Hand-building windowed state, including expiry and out-of-order records. It works, and it is reimplementing Flink badly.
- B

Flink with a sliding one-minute window
Rolling window over a keyed stream is exactly what Flink exists for. State, expiry and ordering are handled.
- C

Firehose to S3, query every minute
Buffering plus query time cannot meet two seconds. Batch latency against a real-time requirement.
- D

Redshift materialized view
A warehouse refreshed on a schedule. Minutes of latency at best, for an alerting requirement.

THE SIGNAL

Rolling one minute period is a sliding window. Two seconds rules out anything that batches first.

Streaming from DynamoDB

- ◆ **DynamoDB Streams.** Item-level changes, 24 hour retention, built in.
- ◆ **Kinesis Data Streams for DynamoDB.** Same changes into a Kinesis stream.
- ◆ Choose Kinesis when you need longer retention or many consumers.
- ◆ Choose DynamoDB Streams for a simple Lambda trigger on change.
- ◆ **Zero-ETL to OpenSearch** removes the pipeline entirely for search use cases.

ON THE EXAM

DynamoDB Streams retains 24 hours and cannot be extended. That limit decides most of these questions.

EventBridge Pipes, the low-code connector

- ◆ **Point to point.** One source, optional filter and enrichment, one target.
- ◆ Sources include Kinesis, MSK, DynamoDB Streams, Amazon SQS.
- ◆ **Filtering happens before** the target is invoked, so you do not pay to discard.
- ◆ Removes glue-code Lambda functions that only forward and filter records.
- ◆ Not a stream processor. No windows, no state, no joins.

ON THE EXAM

A Lambda function that exists only to read a stream and forward some records is what Pipes replaces.

Fan-in and fan-out

- ◆ **Fan-in.** Many producers into one stream. Watch partition key distribution.
- ◆ **Fan-out.** One stream, many consumers with different purposes.
- ◆ **Enhanced fan-out** when consumers contend for shard read capacity.
- ◆ **SNS fan-out** when the payload is a notification, not a data stream.
- ◆ Duplicating a stream per consumer is almost always the wrong answer.

ON THE EXAM

The exam guide names fan-in and fan-out explicitly. Both are usually solved by consumer configuration, not by more streams.

Which of these can you replay?

SERVICE	REPLAY
Kinesis Data Streams	Yes. Rewind within the retention window, up to 365 days
Amazon MSK	Yes. Reset consumer group offsets, retention as configured
DynamoDB Streams	Within 24 hours only. Not extendable
Amazon Data Firehose	No. It delivers and forgets. Replay from the source instead
Amazon SQS	No. Consumed messages are deleted

ON THE EXAM

Replayability is named in the exam guide. It is the property that separates a log from a queue or a delivery service.

What each streaming service charges for

SERVICE	BILLING UNIT
Kinesis, provisioned	Shard-hours, plus PUT payload units. Billed whether used or not
Kinesis, on-demand	Per GB written and read, plus a per-stream hourly charge
Enhanced fan-out	Per consumer-shard hour, plus data retrieved
Data Firehose	Per GB ingested. Extra for format conversion and dynamic partitioning
MSK Provisioned	Broker-hours plus storage. Minimum three brokers in production
MSK Serverless	Cluster-hours, partition-hours, and data in and out

ON THE EXAM

MSK Provisioned has a floor of three brokers running continuously. Kinesis has no equivalent minimum.

Streaming symptoms and their causes

SYMPTOM	LIKELY CAUSE
Write throttling, uneven shard usage	Hot partition key
Write throttling, even usage	Genuinely out of capacity. Add shards or go on-demand
Rising iterator age	Consumers falling behind. Scale consumers or use enhanced fan-out
Consumers slowing each other down	Sharing the 2 MB per second standard read limit
Duplicate records downstream	At-least-once delivery plus a non-idempotent consumer
Data missing after an outage	Retention exceeded, or Firehose discarded after 24 hours

ON THE EXAM

Iterator age is the single most useful streaming metric. It measures how far behind the newest consumer is.

Reading a streaming question quickly

- ◆ **What latency is stated?** Sub-second, seconds, or minutes decides most of it.
- ◆ **Is there existing Kafka?** If so, MSK, and stop comparing.
- ◆ **Does it need memory of other records?** If yes, Flink. If no, Lambda.
- ◆ **How many consumers, and do they conflict?** That is the fan-out question.
- ◆ **Deliver or transport?** A destination and no logic means Firehose.

ON THE EXAM

Next: migration and connectivity. DMS, DataSync, Transfer Family, AppFlow and Data Exchange.